Main sketch explained

A plain-language walkthrough of main_nano_esp32.ino

What this document is for

This is the companion to **Appendix B** of the Star Destroyer Nano ESP32 build guide, which is the main sketch (**main_nano_esp32.ino**) itself - it now has comments throughout explaining what each part does. This document does not repeat the build steps; it picks out the handful of parts that are genuinely a bit fiddly - the Bluetooth button protocol, and the maths behind the engine lighting effect - and walks through them slowly, with real numbers plugged in, rather than just a one-line comment. Read it alongside the sketch itself, not instead of it.

How the sketch is organised

Every Arduino sketch has exactly two special functions. **setup()** runs once, the moment the board is powered on or reset - it is where pins are configured, Bluetooth is started, and anything else that only needs doing once happens. **loop()** then runs over and over again, forever, for as long as the board has power - often hundreds of times a second. Almost everything the show does happens inside loop(), which checks the clock and the current state, and calls out to a handful of smaller helper functions to do the actual work. The table below is the map of those helper functions.

Function	Called from	What it does
setup()	automatically, once	Configures pins, starts Bluetooth, randomises the shimmer and hull order.
loop()	automatically, forever	Checks for button presses, keeps Bluetooth advertising, and runs the show timeline.
feedByte()	the Bluetooth RX callback	Collects incoming Bluetooth bytes into a packet and decodes button presses.
startShow() / stopShow()	loop()	Switch the show into the RUNNING or FADING state.
updateEngines()	loop(), while RUNNING	Works out each engine's brightness, about 50 times a second.
updateFade()	loop(), while FADING	Winds the engines down to zero after OFF is pressed.
shuffleHull()	startShow()	Picks a random order for the 10 hull zones to light up in.

The show is always in exactly one of three states, held in the variable **state**:

State	Meaning
IDLE	Everything is off. Waiting for button 1 (ON) to be pressed.
RUNNING	The full show is playing: engines spooling up and breathing, zones lighting up over time.
FADING	Button 2 (OFF) was just pressed. Zones are already off; the engines are fading down to zero before the state returns to IDLE.

Turning a digital pin into a dimmer: PWM

A microcontroller pin can only ever be fully on or fully off - there is no "half on" setting. To make an LED look dimmed anyway, the pin is switched on and off very fast, faster than the eye can follow, and the **fraction of each cycle spent on** (called the **duty cycle**) controls how bright it looks. This technique is called **pulse-width modulation**, or PWM. The ESP32 chip has dedicated PWM hardware (Espressif calls it "LEDC"), so once it is set up, the sketch just has to choose a duty number and the chip handles the actual switching by itself, tens of thousands of times a second, with no further help needed.

The sketch sets this up once, in pwmBegin():

```
const uint32_t PWM_FREQ = 5000;   // 5000 on/off cycles every second
const uint8_t  PWM_BITS = 12;      // duty is stored as a 12-bit number
const uint32_t PWM_MAX  = 4095;    // 2^12 - 1: the brightest possible duty
```

So a duty of 0 is fully off, a duty of 4095 is fully on, and anything in between is some fraction of the way. For example, a duty of 2048 means the pin is on for roughly 2048/4095 (about 50%) of each 1/5000th-of-a-second cycle, and off for the rest - repeated 5000 times every second.

Duty value	Fraction of PWM_MAX	On-time per cycle (at 5 kHz)
0	0%	0 (fully off)
1024	25%	about 0.05 ms on, 0.15 ms off, repeating
2048	50%	about 0.10 ms on, 0.10 ms off, repeating
4095	100%	0.20 ms on, every cycle (fully on)

One wrinkle: a duty of 2048 does not actually look "half as bright" to the human eye as 4095 - our eyes do not perceive brightness in a straight line against duty cycle. That is what the **gamma correction** in updateEngines() and updateFade() is for, covered in detail further on.

The ESP32-S3 has 8 of these PWM channels. The 7 engines use seven of them, and the eighth drives the **accent lights** (pin D9), so they can fade in and out smoothly too. Their brightness is set by **setAccent()**, which uses the same gamma correction as the engines, but with no breathing or shimmer: the accent lights spool up with the same smoothstep curve and then hold a steady glow at ACCENT_LEVEL.

Why two different sets of PWM function names?

Arduino's ESP32 support ("core") has changed its PWM function names between versions - older installs (core 2.x, which the Nano ESP32 board package is currently based on) use one set of functions, while newer installs (core 3.x) use a different, simpler set. Rather than assuming which one is installed, the sketch checks for a file that only exists on the newer core and picks the matching functions automatically:

```
#if defined(ESP_ARDUINO_VERSION_MAJOR) && (ESP_ARDUINO_VERSION_MAJOR >= 3)
#define PWM_CORE_V3 1
#else
#define PWM_CORE_V3 0
#endif
```

Later, anywhere the sketch needs to call a PWM function, it writes "#if PWM_CORE_V3 ... #else ... #endif" and the compiler quietly includes only the correct half, for whichever core is actually installed. This means the same sketch file works unmodified on either version.

Talking to the phone: the Bluetooth button protocol

The Bluefruit Connect app's "Control Pad" screen is a standard feature of that free app - this sketch does not need its own custom phone app, it just needs to understand the fixed packet format that screen sends. Every time a button is pressed or released, the phone sends exactly 5 bytes:

Byte	Meaning	Example value
0	Start marker - always the character !	'!' (decimal 33)
1	Packet type - always B for a button packet	'B' (decimal 66)
2	Which button, as a text character	'1' (decimal 49) for button 1
3	'1' if the button was just pressed, '0' if just released	'1' (decimal 49)
4	Checksum - lets the receiver detect a corrupted packet	see worked example below

Bluetooth delivers these bytes to the sketch one at a time (sometimes even split across two deliveries, or with two packets stuck together), so the sketch cannot just assume it always receives a neat 5-byte block. Instead, **feedByte()** is called once per byte and keeps a small running buffer, only acting once a complete, correctly-shaped packet has arrived.

Worked example: pressing button 1

Suppose the phone's button 1 is pressed. The five bytes sent, as text characters, are: **! B 1 1 *checksum***. As plain numbers (each character's ASCII value), the first four are 33, 66, 49, 49. The checksum byte exists purely so the receiving end can spot a packet that arrived corrupted - it is calculated by adding the first four bytes together and then flipping every bit of the result (a bitwise NOT, written ~ in C):

Step	Calculation	Result
Add the first four bytes	33 + 66 + 49 + 49	197
Flip every bit (bitwise NOT) of a single byte	255 - 197	58
So the full 5-byte packet is	33, 66, 49, 49, 58	(hex: 21 42 31 31 3A)

feedByte() repeats exactly this calculation on whatever bytes it has collected, and only accepts the packet if its own checksum matches the one the phone sent:

```
uint8_t sum = buf[0] + buf[1] + buf[2] + buf[3];      // 33 + 66 + 49 + 49 = 197
if ((uint8_t)~sum == buf[4] && buf[3] == '1') {       // (uint8_t)~197 = 58 - matches!
  if (buf[2] == '1') pendingCmd = 1;                  // button 1 -> ON
  else if (buf[2] == '2') pendingCmd = 2;             // button 2 -> OFF
}
```

Why check buf[3] == '1' as well? Because the app sends a packet both when a button is **pressed** (byte 3 = '1') and when it is **released** (byte 3 = '0'). Without this check, letting go of the button would immediately re-trigger whatever pressing it just did. Checking for '1' means the sketch reacts only to the press itself.

Why collect bytes at all, instead of just reading 5 bytes in one go?

Bluetooth Low Energy delivers data in whatever chunks the phone's Bluetooth stack happens to send it in - that might be the whole 5-byte packet at once, or it might be split into smaller pieces. feedByte() handles this safely by never assuming a full packet has arrived until it has actually counted 5 bytes starting from a '!', discarding anything that does not fit that shape along the way (for example, if the second byte is not 'B', it gives up and waits for the next '!').

Animating the engines: the brightness formula

updateEngines() runs about 50 times a second (every UPDATE_MS = 20 ms) while the show is RUNNING, and works out a fresh brightness for each of the 7 engines every time. The brightness is built up from three separate effects that are added together, and then one correction that is applied right at the end. Each is simple on its own - it is only the combination that looks complicated at first glance.

Effect 1 - the spool-up ramp

Once an engine's start time (its entry in CH_START_MS) has arrived, it should not simply snap to full brightness - it should ease in smoothly over RAMP_MS (3 seconds). A plain linear ramp (brightness = time elapsed / 3 seconds) would work, but it looks slightly mechanical: it starts and stops moving abruptly. Instead the sketch uses a standard easing curve called **smoothstep**: *ramp = x × x × (3 - 2x)*, where x is how far through the 3 seconds we are (0.0 to 1.0). This accelerates gently at the start and settles gently at the end, instead of moving at a constant speed throughout.

x (fraction of RAMP_MS elapsed)	Straight line (x)	Smoothstep (x × x × (3-2x))
0.00	0.00	0.00
0.25	0.25	0.156
0.50	0.50	0.500
0.75	0.75	0.844
1.00	1.00	1.00

Notice the smoothstep column changes more slowly near the start and end, and faster through the middle - that is what gives the "easing" feel rather than a constant speed.

Effect 2 - breathing

Once fully spooled up, the engines do not sit at a perfectly flat brightness - they slowly rise and fall together, like breathing, using a standard sine wave shared by every engine: *breath = BREATH_DEPTH × sin(2π × time / BREATH_PERIOD_S)*. With the defaults (depth 0.15, period 6 seconds), this gently adds and subtracts up to 0.15 from the brightness level, completing one full rise-and-fall cycle every 6 seconds.

Effect 3 - shimmer

On top of the shared breathing, each engine also gets its own small, fast, slightly irregular flicker, so that all 7 engines do not look identical. This is done by blending two sine waves of different speed for each engine (0.6 of one plus 0.4 of the other), using random frequencies and starting points that setup() picked once at power-up and stored in the per-engine arrays f1, f2, p1, p2. Because every engine has its own random values, no two engines flicker in exact sync, which is what makes it look like independent, natural flicker rather than a repeating pattern.

Combining the three effects

The three are combined as: *level = ramp × (BASE + breath + shimmer)*. Multiplying by ramp (rather than just adding it in) is what makes the whole brightness - breathing, shimmer and all - fade in together during the spool-up, instead of the breathing and shimmer appearing at full strength immediately while only the base brightness ramps up.

Effect 4 - gamma correction (not an effect, a correction)

The level calculated above is a tidy number between 0.0 and 1.0, but sending it to setEngine() unchanged would look wrong. Human eyes are far more sensitive to differences between dim brightnesses than between bright ones - a change from a PWM duty of 10% to 20% looks like a much bigger jump than a change from 80% to 90%, even though both are "10% more duty". **Gamma correction** compensates for this by reshaping the 0.0-1.0 level with *pow(level, GAMMA)* (GAMMA = 2.2, a very standard value for this kind of correction) before turning it into a duty value, which pulls the middle brightnesses down more than it pulls down the bright end:

level (0.0-1.0)	Without gamma: level × 4095	With gamma: level to the power 2.2, × 4095
0.00	0	0
0.25	1024	193
0.50	2048	891
0.75	3071	2174
1.00	4095	4095

Notice the two columns agree at the very top and bottom, but the gamma-corrected duty is much lower than the plain one in the middle - that lower value is what actually looks evenly "half as bright" to a human eye, even though the number itself is far from half of 4095. This is exactly the same correction used in updateFade() when the engines wind back down, so a fade-out looks smooth and even rather than appearing to "drop off a cliff" near the end.

Putting it all together, here is the full calculation from updateEngines(), annotated:

```
float x = (now - CH_START_MS[i]) / (float)RAMP_MS;        // Effect 1: how far through the spool-up
float ramp = x * x * (3.0 - 2.0 * x);                     //           smoothstep-eased
float shimmer = SHIMMER * (0.6*sin(...) + 0.4*sin(...));  // Effect 3: this engine's own flicker
float level = ramp * (BASE + breath + shimmer);           // Effects 1+2+3 combined
setEngine(i, pow(level, GAMMA) * PWM_MAX);                // correction, then convert to a duty value
```

Picking a random order: shuffling the hull zones

The 10 hull zones light up one at a time over the course of the show, at random gaps apart, but in a different order every time the show runs - this is done by shuffleHull(), using a well-known, reliable algorithm called the **Fisher-Yates shuffle**. It starts with the zones in their normal order (0, 1, 2, ...) and then walks backwards through the list once, swapping each position with a random earlier-or-equal position as it goes.

Worked example with just 4 zones (the real sketch has 10, but the idea is identical)

Start: order = [0, 1, 2, 3]

Step (i)	Random j (0 to i)	Swap	order[] afterwards
i = 3	say j = 1	swap positions 3 and 1	[0, 3, 2, 1]
i = 2	say j = 2	swap positions 2 and 2 (no change)	[0, 3, 2, 1]
i = 1	say j = 0	swap positions 1 and 0	[3, 0, 2, 1]

The final order, [3, 0, 2, 1], means: zone 3 lights up first, then zone 0, then zone 2, then zone 1. Because the random numbers are different every time the show starts (see "Why the pattern is different every time" below), the order comes out different on every run.

Why the pattern is different every time

random() on its own would actually produce the exact same sequence of "random" numbers every time the board is reset, unless it is given a different starting point (a "seed") each time. setup() creates that starting point with *randomSeed(esp_random() ^ micros())*: esp_random() reads the ESP32's built-in hardware random number generator, and micros() (the number of microseconds the board has been running) depends on the exact moment it was switched on. Combining the two (with the bitwise XOR operator, ^) gives a seed that is, for practical purposes, different every single time - so the hull order, and the exact shimmer flicker pattern, is never quite the same twice.

The show timeline: how loop() decides when things happen

Every timed event in the show - an engine starting, the bridge or hangar bay lighting up, each hull zone appearing - is scheduled as "this many milliseconds after the ON button was pressed", not as a real clock time. The moment ON is pressed, startShow() records that instant in **t0** (using millis(), the number of milliseconds the board has been running). From then on, loop() simply checks, on every single pass, whether *millis() - t0* has reached each event's scheduled time yet.

Time after ON	Event
0.0 s	Engine 1 starts its 3-second spool-up
2.5 s	Engine 2 starts its spool-up
5.0 s	Engine 3 starts its spool-up
8.0 s	Engines 4, 5, 6 and 7 all start their spool-up together
12.5 s	Bridge lights switch on; the first hull zone's reveal time is picked (10-30 s from now)
14.0 s	Accent lights start a smooth 4-second fade up to a steady glow
16.5 s	Hangar bay lights switch on
varies	Hull zones appear one at a time, 10-30 s apart (randomly, re-picked after each one), until all 10 are lit
2 hours (default)	The show switches itself off automatically, fading the engines and accent lights out first

A flag per event (bridgeOn, hangarOn, and the count nextHull) makes sure each one only fires once, the first time its scheduled moment is reached, rather than re-triggering on every single pass through loop() after that moment has passed.

Why millis() instead of delay()?

delay(5000) would pause the entire sketch for 5 seconds, during which nothing else could happen - no button presses would be noticed, and the Bluetooth connection could even time out. Checking millis() instead lets loop() keep running continuously, thousands of times a second, so a button press or a Bluetooth event is noticed almost instantly no matter what else the show is doing at that moment. This is the standard way to handle timing in any Arduino sketch that also needs to stay responsive, and it is used throughout this one.

Keeping the Bluetooth connection available

One more piece of timeline-style bookkeeping happens on every pass through loop(), regardless of the show's state: once a phone disconnects, the board must explicitly call startAdvertising() again, or it would simply vanish from Bluetooth and nothing could ever reconnect to it. loop() watches the bleConnected flag (set by Bluetooth's own callback functions) and restarts advertising the moment a disconnection is noticed.